
Izveštaj o eksperimentima jakog i slabog skaliranja

Paralelni algoritmi na grafovima — BFS (Breadth-First Search)

Analiza performansi paralelizacije BFS algoritma na retkim, nasumično generisanim neusmerenim grafovima. Poređenje eksperimentalnih rezultata sa teorijskim predviđanjima Amdahlovog i Gustafsonovog zakona, uz implementacije u Python-u (multiprocessing) i Rust-u (rayon).

Implementacije: Python 3.11 (multiprocessing) • Rust 1.95 (rayon)

Metod merenja: 30 ponovljenih merenja po konfiguraciji

1. Tehnički detalji sistema

Hardver

Komponenta	Specifikacija
Procesor	12th Gen Intel Core i5-1235U
Takt	Bazni 1300 MHz (boost do 4.4 GHz)
Fizička jezgra	10 (2 P-core + 8 E-core)
Logička jezgra	12
Cache	L1 80 KB/jezgro · L2 1.25 MB (P-core) / 2 MB (E-core) · L3 12 MB shared
NUMA	1 node
RAM	16 GB
Operativni sistem	Microsoft Windows 11 Pro, verzija 10.0.26100

Softver i biblioteke

Alat / biblioteka	Verzija	Namena
Python	3.11.0	Referentna implementacija
Rust (rustc)	1.95.0 (59807616e, 2026-04-14)	Kompajlirana implementacija
Cargo	1.95.0	Build sistem za Rust
rayon	1.10	Rust paralelizacija
plotters	0.3	Rust vizualizacija
matplotlib	3.10.7	Vizualizacija (Python)
pandas	2.3.3	Obrada podataka
numpy	2.2.6	Numeričke operacije

2. Problem i implementacija

Analiziran je BFS (Breadth-First Search) algoritam na nasumično generisanim neusmerenim grafovima sa gustinom ivica $p = 0.001$. Cilj je bio ispitati koliko se ovaj klasičan algoritam može ubrzati paralelizacijom i uporediti izmerene performanse sa teorijskim granicama.

Sekvencijalni deo koda

BFS je inherentno sekvencijalan algoritam — svaki nivo (level) pretrage zavisi od rezultata prethodnog nivoa i ne može se izračunati pre njega. Procenjeni procenat sekvencijalnog dela: ~95% ($f_{seq} = 0.95$).

Paralelni deo koda

Jedino što se može paralelizovati jeste procesiranje čvorova unutar jednog nivoa (level-synchronous BFS). Procenjeni procenat paralelnog dela: ~5% ($f_{par} = 0.05$).

3. Teorijski maksimumi ubrzanja

Amdahlov zakon (jako skaliranje)

Formula: $S(p) = 1 / (f_{seq} + f_{par} / p)$

gde su $f_{seq} = 0.95$, $f_{par} = 0.05$, a p = broj jezgara.

Broj jezgara p	Teorijsko ubrzanje $S(p)$
1	1.000
2	1.026
4	1.050
8	1.075
∞ (beskonačno)	$S_{max} = 1 / 0.95 \approx 1.053$

Gustafsonov zakon (slabo skaliranje)

Formula: $S(p) = p - f_{seq} \times (p - 1)$

gde je $f_{seq} = 0.95$.

Broj jezgara p	Teorijska efikasnost E
1	1.000
2	0.525
4	0.288
8	0.169

4. Jako skaliranje

Definicija: Veličina problema je fiksna ($n = 20000$ čvorova, $p = 0.001$), a menja se broj procesorskih jezgara. Idealno bi vreme izvršavanja opadalo srazmerno broju jezgara.

Python multiprocessing — jako skaliranje (30 merenja po kombinaciji)

Workers	Srednje vreme (s)	Std. dev. (s)	Ubrzanje	Teor. max (Amdahl)
1	0.004756	0.000460	1.000	1.000
2	0.346846	0.027705	0.014	1.026
4	0.695850	0.161089	0.007	1.050
8	1.377423	0.090710	0.003	1.075

Outlieri: Kod workers = 4 primećena je visoka standardna devijacija (0.161 s) što ukazuje na nestabilnost merenja usled Windows process scheduling overhead-a.

Rust rayon — jako skaliranje (30 merenja po kombinaciji)

Threads	Srednje vreme (s)	Std. dev. (s)	Ubrzanje	Teor. max (Amdahl)
1	0.004030	0.001023	1.000	1.000
2	0.047814	0.006260	0.084	1.026
4	0.040523	0.002989	0.099	1.050
8	0.040299	0.004823	0.100	1.075

Outlieri: Nisu detektovani značajni outlieri. Standardna devijacija je konzistentna kroz sve konfiguracije.

Zaključak jakog skaliranja

Izmereno ubrzanje je drastično ispod teorijskog Amdahlovog maksimuma u oba slučaja. Kod Python-a, overhead multiprocessing.Pool (fork, IPC, serijalizacija podataka) potpuno dominira nad korisnim radom. Kod Rust-a, rayon thread overhead i memory contention nadmašuju dobitak od paralelizacije. Ovo potvrđuje da je BFS na retkim grafovima loš kandidat za paralelizaciju level-synchronous pristupom.

5. Slabo skaliranje

Definicija: Veličina problema raste proporcionalno broju jezgara — svako jezgro dobija konstantan posao. Bazna veličina: $n = 5000$ čvorova po jezgru.

Workers	n (čvorova)	Posao po jezgru
1	5000	5000
2	10000	5000
4	20000	5000
8	40000	5000

Python multiprocessing — slabo skaliranje

Workers	n	Srednje vreme (s)	Std. dev. (s)	Efikasnost	Teor. (Gustafson)
1	5000	0.002900	0.000400	1.000	1.000
2	10000	0.875200	0.100800	0.003	0.525
4	20000	2.483300	0.269800	0.001	0.288

Napomena: Merenje za workers = 8 nije završeno usled zamrzavanja Python multiprocessing procesa na Windows-u (poznati problem sa Windows fork semantikom).

Rust rayon — slabo skaliranje (30 merenja po kombinaciji)

Threads	n	Srednje vreme (s)	Std. dev. (s)	Efikasnost	Teor. (Gustafson)
1	5000	0.000308	0.000093	1.000	1.000
2	10000	0.015381	0.001407	0.020	0.525
4	20000	0.034783	0.004119	0.009	0.288
8	40000	0.090026	0.010485	0.003	0.169

Outlieri: Kod threads = 1 primećena je relativno visoka devijacija (0.000093 s) u odnosu na srednje vreme (0.000308 s) — ~30% — što je posledica mernog šuma pri veoma kratkim vremenima izvršavanja.

Zaključak slabog skaliranja

Efikasnost drastično opada sa brojem jezgara, što je daleko ispod Gustafsonovog teorijskog minimuma. Ovo je direktna posledica toga što overhead komunikacije između threadova raste brže od veličine problema. BFS na retkim grafovima nema dovoljno paralelnog posla po jezgru da kompenzuje overhead.

6. Opšti zaključak

Oba eksperimenta (jako i slabo skaliranje) konzistentno pokazuju da naivna level-synchronous BFS paralelizacija nije efikasna na retkim grafovima. Glavni razlozi su:

1. Visok sekvencijalni deo koda (~95%) — Amdahlov zakon ograničava teorijsko ubrzanje na maksimalno 1.053x bez obzira na broj jezgara.
2. Mali frontieri — Retki grafovi ($p = 0.001$) imaju malo čvorova po nivou, pa nema dovoljno posla za threadove.
3. Overhead paralelizacije — Thread spawn, sinhronizacija i memory contention dominiraju nad korisnim radom.
4. Python GIL i IPC — Python multiprocessing zahteva serijalizaciju podataka između procesa, što je posebno skupo za grafovske strukture.

Rust implementacija je sekvencijalno 7–38x brža od Python-a, ali paralelna verzija ne donosi ubrzanje ni u jednom jeziku za ovaj tip problema. Za praktičnu paralelizaciju BFS-a na ovakvim grafovima bili bi neophodni napredniji pristupi (npr. direction-optimizing BFS ili grupno procesiranje više nezavisnih izvora).